| | |
|---|---|
| **Started on** | Thursday, 9 April 2026, 11:12 AM |
| **State** | Finished |
| **Completed on** | Thursday, 9 April 2026, 12:52 PM |
| **Time taken** | 1 hour 40 mins |
| **Grade** | **0.00** out of 400.00 (**0**%) |

Question **1**

Not answered

Marked out of 100.00

| | |
|---|---|
| Time limit | 1 s |
| Memory limit | 64 MB |

# CipherCloud

Anda adalah seorang Software Architect di **CipherCloud**, sebuah perusahaan infrastruktur cloud global. Sistem lama sering mengalami kebocoran data dan salah memberikan akses kepada publik karena hierarki dokumen yang berantakan.

CEO akhirnya menugaskan Anda untuk merombak total dua komponen paling krusial: **CloudNode** dan **Hierarki Record**. Arsitektur harus dibuat seketat mungkin agar developer lain tidak dapat secara sembarangan menyalahgunakan fungsi sistem.

**Link Codebase. Anda dilarang memodifikasi file header `BaseRecord.hpp`, `main.cpp`, dan `Formatter.hpp` yang telah disediakan.**

## Bagian 1: Manajemen `CloudNode`

Anda diminta untuk melengkapi deklarasi dan implementasi kelas `CloudNode`, dengan mematuhi spesifikasi berikut:

| Komponen | Spesifikasi |
|---|---|
| **Atribut** | • `server_name` (`string`): Nama dari node cloud.<br>• `used_gb` (`int`): Jumlah GB yang sudah digunakan.<br>• `limit_gb` (`int`): Batas maksimal GB yang dapat digunakan. |
| **Constructor** | Menerima parameter `nama server` dan `limit GB`, kemudian inisialisasi `used_gb` ke 0. Print pesan constructor dari `Formatter`. |
| **Copy Constructor** | Membuat backup dari `CloudNode` (other). Nama server diikuti `"_backup"` (append server_name other), `limit_gb` bernilai sama dari sumber asal, dan `used_gb` diinisialisasi ke 0. Print pesan copy constructor dari `Formatter`. |
| **Copy Assignment Operator** | Menyalin `used_gb` dari `CloudNode` lain ditambah 2, tetapi menyalinnya tidak boleh melebihi `limit_gb` target. Nilai `limit_gb` target dan tipe nama server **tidak berubah**. Print pesan assignment operator dari `Formatter` sebelum ekspresi return. |
| **Destructor** | Print pesan destructor dari `Formatter`. |
| **Operator `+` (Int)** | Menerima sebuah angka (int). Menambah `limit_gb` sebanyak n GB, dan mengembalikan `CloudNode` baru dengan perubahan tersebut. |
| **Operator `-` (Int)** | Menerima sebuah angka (int). Mengurangi `used_gb` sebanyak n GB, tidak boleh kurang dari 0 (set ke 0 jika hasilnya negatif). Mengembalikan `CloudNode` baru dengan perubahan tersebut. |
| **Friend Function** | `friend void systemWipe(CloudNode &node)`: Menerima referensi ke `CloudNode`. Mengatur `used_gb` dan `limit_gb` node target seketika menjadi 0 tanpa mengubah namanya. |

## Bagian 2: `Record Hierarchy`

Telah disediakan kelas dasar `BaseRecord`. Rancang dan implementasikan (buat file `.hpp` dan `.cpp`) kelas-kelas turunan berikut. Konstruktor dari setiap kelas turunan wajib meneruskan parameter `author` dan `key` ke konstruktor `BaseRecord`. Pahami deskripsi arsitektur berikut untuk menentukan **Access Modifier Inheritance** yang tepat. Spesifikasi:

| Nama Kelas | Spesifikasi Pewarisan & Sifat Arsitektur | Daftar Implementasi Metode |
|---|---|---|
| `PublicRecord` | Semua fungsionalitas dan kode eksternal yang menerima `BaseRecord` wajib **dapat menerima objek `PublicRecord` sebagai penggantinya** (Dikenali secara eksplisit oleh dunia luar sebagai `BaseRecord`). | • **Constructor:** Meneruskan parameter secara standar.<br>• **`calculateClearance()`:** Lakukan **override** pada fungsi ini sehingga selalu mereturn nilai konstanta `0`. |
| `InternalRecord` | Identitas kelas ini **tidak boleh diekspos sebagai `BaseRecord` ke sistem luar/publik**. Akan tetapi, struktur harus *extensible*, developer di masa depan harus tetap bisa membuat subclass dari kelas ini, dan subclass tersebut **harus tetap bisa mewarisi & mengakses fungsionalitas dari `BaseRecord`**. | • **Constructor:** Meneruskan parameter secara standar.<br>• **`peekSecurity()`:** Fungsi baru (publik) spesifik kelas ini yang mengembalikan nilai `calculateClearance()` milik turunan, lalu dikalikan `2`. |
| `SecureVaultRecord` | Identitas kelas ini **tidak boleh diekspos sebagai `BaseRecord` ke sistem luar/publik**. Akan tetapi, struktur tidak *extensible*. Jika developer di masa depan membuat subclass dari kelas ini, subclass tersebut **tidak bisa mewarisi & mengakses fungsionalitas dari `BaseRecord`**. | • **Constructor:** Meneruskan parameter secara standar.<br>• *Tidak diperlukan fungsi/metode khusus tambahan.* |

**Contoh Keluaran main.cpp:**

```
 CLOUD NODE BEHAVIOUR
 [CTOR] Node NodeA diinisialisasi. NodeA - used GB: 0, limit GB: 10
 [CCTOR] Backup NodeA_backup dibuat. NodeA_backup - used GB: 0, limit GB: 15
 [CTOR] Node NodeC diinisialisasi.
 [ASSIGN] Node NodeC disinkronisasi.
 [CCTOR] Backup NodeC_backup dibuat. NodeC - used GB: 2, limit GB: 50 NodeC - used GB: 0, limit GB:
0
 [DTOR] Node NodeC_backup dinonaktifkan.
 [DTOR] Node NodeC dinonaktifkan.
 [DTOR] Node NodeA_backup dinonaktifkan.
 [DTOR] Node NodeA dinonaktifkan.
 RECORD HIERARCHY BEHAVIOUR
 [PASS] PublicRecord mengembalikan clearance 0.
 [PASS] InternalRecord mengembalikan sekuritas operasional yang tepat (10).
 [PASS] SecureVaultRecord berhasil dibentuk dan dikompilasi!
```

Note: Jika Anda mendapatkan Runtime Error saat proses grading di olympia, perhatikan kembali penerapan prinsip enkapsulasi dan pemilihan Access Modifier Inheritance

## Format Pengumpulan

Kumpulkan file-file berikut:

`CloudNode.hpp`, `CloudNode.cpp`, `PublicRecord.hpp`, `PublicRecord.cpp`, `InternalRecord.hpp`, `InternalRecord.cpp`, `SecureVaultRecord.hpp`, `SecureVaultRecord.cpp`

Satukan ke dalam **satu file zip** (nama file zip dibebaskan).

C++14

Question **2**

Not answered

Marked out of 100.00

| | |
|---|---|
| Time limit | 1 s |
| Memory limit | 64 MB |

# Football Manager

## Deskripsi Soal

Sebuah klub sepak bola profesional membutuhkan sistem manajemen anggota yang mampu merepresentasikan berbagai peran di dalam klub: pemain lapangan, penjaga gawang, pelatih kepala, fisioterapis, dan pemain-pelatih.

**Kamu diminta untuk mengimplementasikan kelas ClubMember, Player, Staff, serta PlayerCoach dari hierarki kelas berikut. Selain itu, kamu juga perlu mengimplementasikan method di kelas Club.**

Link Codebase

Petunjuk: Gunakan kelas Formatter untuk formatting output.

## Hierarki Kelas

```
            ClubMember
           /          \
       Player          Staff
         |   \        /   |
   Goalkeeper  PlayerCoach  HeadCoach
 OutfieldPlayer             Physio
```

## Spesifikasi Kelas

### `ClubMember`

Atribut `protected`:

- `string name`
- `int age`
- `string contractEnd`

Method `public`:

- Konstruktor: `ClubMember(name, age, contractEnd)`
- Pure virtual: `string getProfile()`
- Pure virtual: `double calculateWage()`
- Pure virtual: `void work()`
- Destruktor - mencetak: `[~ClubMember] <name> contract record cleared.`

### `Player`

Atribut `protected` tambahan:

- `string position`
- `int stamina`
- `double rating`

Method `public`:

- Konstruktor: `Player(name, age, contractEnd, position, stamina, rating)`
- Method: `double calculateWage()` dengan rumus `rating * 10000.0`
- Pure virtual: `double calculateRating()`
- Destuktor - mencetak: `[~Player] <name> training log freed.`

### `Staff`

Atribut `protected` tambahan:

- `string coachingLicense`
- `string department`

Method `public`:

- Konstruktor: `Staff(name, age, contractEnd, license, dept)`
- Method: `double calculateWage()` dengan fixed return `50000.0`
- Pure virtual: `string getSpecialty()`
- Destruktor - mencetak: `[~Staff] <name> session record freed.`

### `PlayerCoach`

Atribut `private` tambahan: `int yearsAsPlayer`, `bool isCurrentlyPlaying`

- Konstruktor: `PlayerCoach(name, age, contractEnd, position, stamina, rating, license, yearsAsPlayer, isCurrentlyPlaying)` - department `"Coaching"`
- `void work()` jika `isCurrentlyPlaying`: `[PlayerCoach] <name> plays AND coaches from the pitch.`; jika tidak: `[PlayerCoach] <name> coaches from the touchline.`
- `double calculateRating()` dengan rumus `rating * 0.7 + (yearsAsPlayer / 20.0) * 3.0`
- `std::string getSpecialty()` dengan output `"Playing + Coaching (Player-Coach, <years> yrs exp)"`
- `double calculateWage()` → Penjulamahan dari calculateWage Player dan Staff (HARUS MEMANGGIL FUNGSI, TIDAK BOLEH DIHARDCODE!)
- `std::string getProfile()` → format: `PlayerCoach | <name> | Pos: <pos> | Lic: <license> | Specialty: <specialty> | Wage: $<W>/wk`
- Destruktor → `[~PlayerCoach] <name> dual-role contract terminated.`

### `Club`

Atribut `private`: `std::string clubName`, `std::vector<ClubMember*> roster`

- Konstruktor: `Club(name)`
- `void addMember(ClubMember* member)` - menambahkan anggota ke roster
- `void printSquadReport() const` - mencetak header dan profil semua anggota
- `void runTraining() const` - mencetak header dan memanggil `work()` semua anggota
- Destruktor - mencetak header dan `delete` semua pointer

---

## Kelas Pembantu — `Formatter`

| Method | Deskripsi |
|---|---|
| `Formatter::wage(double)` | Menghasilkan string `$<nilai>/wk` (nilai dibulatkan ke integer) |
| `Formatter::fixed(double, int)` | Menghasilkan string angka dengan jumlah desimal tertentu |
| `Formatter::makeRow(label, name, fields)` | Membangun baris profil dengan label di-pad ke 15 karakter |
| `Formatter::log(label, name, message)` | Mencetak `[label] name message` ke stdout |

---

## Berkas Pengumpulan

**Kumpulkan berkas-berkas dibawah ini dalam 1 zip!**

| Berkas |
|---|
| `ClubMember.hpp / .cpp` |
| `Player.hpp / .cpp` |
| `Staff.hpp / .cpp` |
| `PlayerCoach.hpp / .cpp` |
| `Club.hpp / .cpp` |

C++14

Question **3**

Not answered

Marked out of 100.00

| | |
|---|---|
| Time limit | 1 s |
| Memory limit | 64 MB |

**Nama File**: SafeBox.zip

Para Nimon membutuhkan sistem penyimpanan data sementara yang fleksibel. Sistem ini harus mampu menyimpan berbagai tipe data, bilangan bulat, teks, maupun tipe lainnya, *tanpa perlu membuat ulang kode untuk setiap tipe*. Namun, setiap operasi yang salah harus ditangani dengan baik agar sistem tidak meledak!

Kamu diminta mengimplementasikan dua file header yang digunakan oleh **main.cpp** ( **tidak boleh diubah**):

- **SafeBoxException.hpp** yang merupakan hierarki kelas exception untuk SafeBox
- **SafeBox.hpp** yang merupakan template kelas `SafeBox<T>` dan template fungsi `safeMerge`

### Spesifikasi

#### 1. Hierarki SafeBoxException

Buat hierarki exception dengan **SafeBoxException** sebagai base class (turunan dari `std::exception`). Terdapat tiga turunan spesifik untuk kondisi error yang berbeda:

- **BoxFullException** untuk operasi `store` ke box yang sudah penuh.
  Menyimpan kapasitas maks (int).
  `what()` → `"Box penuh: kapasitas maks N"`
- **BoxEmptyException** untuk operasi `retrieve`, `peek`, dan `reverse` pada box yang kosong.
  `what()` → `"Box kosong"`
- **InvalidItemException<T>** untuk item yang tidak valid merupakan template class sehingga dapat menyimpan item bertipe apa saja.
  Menyimpan item asli bertipe T.
  `what()` → `"Item tidak valid: <item>"` (gunakan `ostringstream` untuk mengonversi T ke string)

#### 2. Template Class SafeBox<T>

Kelas generik `SafeBox<T>` menyimpan elemen secara **LIFO (Last In First Out)** dengan kapasitas terbatas. Implementasikan method-method berikut:

- `SafeBox(int capacity, std::function<bool(const T&)> validator = nullptr)`: Konstruktor dengan kapasitas maksimum dan validator opsional. Jika diberikan, validator dipanggil di dalam `store()` untuk memeriksa keabsahan item sebelum menyimpannya.
- `void store(const T& item)`: Menyimpan item ke dalam box. Lempar `InvalidItemException<T>` jika validator ada dan item tidak lolos validasi. Lempar `BoxFullException` jika penuh
- `T retrieve()`: Mengambil dan menghapus item teratas (LIFO). Lempar `BoxEmptyException` jika kosong
- `T peek() const`: Melihat item teratas tanpa menghapusnya. Lempar `BoxEmptyException` jika kosong
- `void reverse()`: Membalik urutan semua elemen dalam box. Lempar `BoxEmptyException` jika box kosong saat dipanggil
- `int size() const`: Mengembalikan jumlah item saat ini
- `int capacity() const`: Mengembalikan kapasitas maksimum
- `bool isEmpty() const`: Mengembalikan `true` jika box kosong
- `operator<<`: Overloading output stream untuk mencetak isi box dari bawah ke atas dalam format: `[item_bawah, ..., item_atas]`. Jika kosong, cetak `[]`.

Implementasi dibebaskan selama memenuhi spesifikasi. Boleh menggunakan STL (misalnya `std::stack`) untuk penyimpanan internal.

#### 3. Template Function safeMerge

Implementasikan fungsi global bertipe template:

```
int safeMerge(SafeBox<T>& src, SafeBox<T>& dst)
```

Fungsi ini memindahkan item dari `src` ke `dst` satu per satu (urutan LIFO), dengan ketentuan:

- Hentikan pemindahan jika `dst` penuh, tangkap `BoxFullException` secara internal dan berhenti
- Kembalikan jumlah item yang berhasil dipindahkan

---

### Format Masukan

Baris pertama berisi tiga integer `cap1 cap2 cap3`: kapasitas `box1` (int), `box2` (int), dan `sbox` (string). Pada `sbox`, string yang memiliki panjang kurang dari 2 dianggap tidak valid dan akan ditolak oleh validator yang sudah dipasang pada konstruktornya. Baris kedua berisi integer `Q`. Selanjutnya `Q` baris perintah:

- `PUSH n v` - store nilai `v` ke box ke-`n` (n=1: box1 int, n=2: box2 int, n=3: sbox string)
- `POP n` - retrieve dari box ke-`n`

- `PEEK n` - peek box ke-`n`
- `REVERSE n` - reverse box ke-`n`
- `PRINT n` - cetak isi box ke-`n`
- `SIZE n` - cetak ukuran box ke-`n`
- `MERGE` - panggil `safeMerge(box1, box2)`

**Format Keluaran**

| Perintah | Kondisi | Keluaran |
|---|---|---|
| **PUSH 1 / PUSH 2** | berhasil | `Tersimpan: v` |
| **PUSH 1 / PUSH 2** | box penuh | `BoxFullException::what()` |
| **PUSH 3** | berhasil | `Tersimpan: v` |
| **PUSH 3** | panjang < 2 (ditolak validator) | `InvalidItemException<string>::what()` |
| **PUSH 3** | sbox penuh | `BoxFullException::what()` |
| **POP n** | berhasil | `Diambil: v` |
| **POP n** | box kosong | `BoxEmptyException::what()` |
| **PEEK n** | berhasil | nilai teratas |
| **PEEK n** | box kosong | `BoxEmptyException::what()` |
| **REVERSE n** | berhasil | `Dibalik` |
| **REVERSE n** | box kosong | `BoxEmptyException::what()` |
| **PRINT n** | — | `[item_bawah, ..., item_atas]` atau `[]` jika kosong |
| **SIZE n** | — | jumlah item (integer) |
| **MERGE** | — | `Dipindahkan: N item` |

**Contoh Masukan dan Keluaran**

**Masukan 1:**

```
3 3 3 7 PUSH 1 1 PUSH 1 2 PUSH 1 3 PRINT 1 REVERSE 1 PRINT 1 POP 1
```

**Keluaran 1:**

```
Tersimpan: 1 Tersimpan: 2 Tersimpan: 3 [1, 2, 3] Dibalik [3, 2, 1] Diambil: 1
```

**Masukan 2:**

```
2 2 2 10 PUSH 3 A PUSH 3 Hi PUSH 3 OK PUSH 3 Yes PUSH 3 No PRINT 3 PEEK 3 REVERSE 3 PRINT 3 POP 3
```

**Keluaran 2:**

```
Item tidak valid: A Tersimpan: Hi Tersimpan: OK Box penuh: kapasitas maks 2 Box penuh: kapasitas maks 2 [Hi
```

**NOTE: JANGAN MASUKKAN FILE main.cpp KE DALAM ZIP**

C++14

Question **4**

Not answered

Marked out of 100.00

| | |
|---|---|
| Time limit | 1 s |
| Memory limit | 64 MB |

# Deskripsi Soal

Anda adalah seorang Backend Developer di sebuah platform *livestreaming*. Anda diminta untuk melengkapi sistem moderasi otomatis (`CommentsManager`) yang bertugas menyaring pesan, mendeteksi spam, mencari pemenang kuis, dan memprioritaskan chat dari penonton VIP. Sistem ini bergantung pada kelas `Comment` yang merepresentasikan setiap pesan yang masuk.

# Tugas Anda

Anda diwajibkan untuk menggunakan STL C++ dan memanfaatkan Lambda Expressions. **Penggunaan `for/while` loop sangat dilarang dan tidak akan dinilai!**

Berdasarkan `CommentsManager.hpp`, buatlah implementasi untuk 4 fungsi berikut:

1. `void CommentsManager::kickSpammer(int threshold_score)`
2. `void CommentsManager::printViolation()`
3. `std::string CommentsManager::quizWinner(const std::set<std::string>& passwords)`
4. `void CommentsManager::upVIPComment()`

Silakan buka dokumentasi cppreference atau handbook (lebih ringkas) berikut untuk keperluan STL. Pastikan anda menggunakan referensi sesuai dengan C++14 ke bawah agar bisa dijalankan olympia.

Berikut adalah file `main.cpp` dan `outputnya`

---

# Berkas Pengumpulan

Anda hanya perlu mengumpulkan file `CommentsManager.cpp`

C++14

◄ Post Praktikum 3

Jump to...